התחיל ב:	19:14 ,29/07/2022
מצב	הסתיים
הושלם ב-	16:24 ,3/08/2022
הזמן שלקח	4 ימים 21 שעות
איחר את המועד	4 ימים 18 שעות

שאלה **1**

לא נענה

ניקוד השאלה: 10.00

6) A. נק' רשמו את סיבוכיות המקום (Space Complexity) וסיבוכיות זמן הריצה (Time Complexity) של הפונקציה הבאה, **f1** כפונקציה של **n** ושל **m**:

```
def f1(n, m):
    liz = [[0]*m for i in range(n)]
    for i in range(n):
        for j in range(m):
            k = 1
            while k < len(liz)*len(liz[0]):
                k *= 2
                a,b=(n-1)%(i+2*j+k), (m-1)%(2*i+j+k)
                liz[a][b] += 1
```

השתמשו בסימונים המקובלים לזמני ריצה. יש לכתוב את התשובה ללא רווחים מיותרים.

Space Complexity:

Time Complexity:

---

4) B. נק' בחרו את סיבוכיות זמן הריצה של הפונקציה הבאה, **f2**, כפונקציה של **n**:

```
def f2(n):
    i, j, k, x = 0, 0, 0, 0
    for i in range(n+1):
        for j in range(i+1, n+1):
            for k in range(1, j+1):
                x+=1
    return x
```

שאלה **2**

לא נענה

ניקוד השאלה: 10.00

להלן הגדרות המחלקה **Lnk**, אשר מייצגת רשימה מקושרת:

```
class Lnk:
    empty = ()
    def __init__(self, data, next = empty):
        self.data = data
        self.next = next
    def __repr__(self):
        if self.next is Lnk.empty:
            return "Lnk({})".format(self.data)
        else:
            return "Lnk({}, {})".format(self.data, self.next)
```

---

4) A. נק') השלימו במקומות המסומנים את הקוד של הגנרטור הרקורסיבי **go_back(lg)**, אשר מקבל רשימה מקושרת **lg** ומחזיר את אברי הרשימה בסדר הפוך.

דוגמא:

```
>>> list(go_back(Lnk(2, Lnk((23,), Lnk("Ety")))))
['Ety', (23,), 2]
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def go_back(lg):
    if lg:
        yield ______(1)______
        yield ______(2)______
```

(1)

(2)

---

6) B. נק') הפונקציה **swap_next(lng)** (לא רקורסיבית) מקבלת רשימה מקושרת **lng**, ומחליפה את הערכים בין זוגות האיברים משמאל לימין. רשימות עם איבר אחד ורשימות ריקות נשארות ללא שינוי.

דוגמאות:

```
>>> lnz = Lnk(7, Lnk(1, Lnk(5, Lnk(3, Lnk(8,Lnk(-3, Lnk(6)))))))
>>> swap_next(lnz)
>>> lnz
Lnk(1, Lnk(7, Lnk(3, Lnk(5, Lnk(-3, Lnk(8, Lnk(6)))))))

>>> one_el = Lnk(8)
>>> swap_next(one_el)
>>>one_el
Lnk(8)

>>> empt = Lnk.empty
>>> swap_next(empt)
>>> empt
()
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def swap_next(lng):
    while ______(1)______:
        ______(2)______ = ______(3)______
        lng = ______(4)______
```

(1)

(2)

(3)

(4)

שאלה **3**

לא נענה

ניקוד השאלה: 10.00

5) A. נק' הפונקציה הרקורסיבית **`add_pos_evens(liz)`** מקבלת רשימה של שלמים **`liz`**, ומחזירה את סכום השלמים הזוגיים החיוביים שברשימה **`liz`**.

דוגמאות:

```
>>> add_pos_evens([])
0

>>> add_pos_evens([1,2,3,4,10])
16

>>> add_pos_evens([-2,-7,-8, 5])
0
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def add_pos_evens(liz):
    if len(liz) == 0:
        return 0
    else:
        if liz[0] % 2 == 0 and liz[0] >= 0:
            return ______(1)______ + ______(2)______
        else:
            return ______(3)______
```

(1)

(2)

(3)

---

5) B. נק' הפונקציה הרקורסיבית **`high_low(n)`** מקבלת מספר שלם חיובי **`n`**, ומחזירה **`True`** אם הספרות של **`n`** בסדר מונוטוני יורד משמאל לימין, ו-**`False`** אחרת.

דוגמאות:

```
>>> high_low(2)
True

>>> high_low(70)
True

>>> high_low(111)
True

>>> high_low(112)
False

>>> high_low(7674)
False
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def high_low(n):
    if ______(1)______:
        return True
    elif ______(2______:
        return False
    else:
        return ______(3)______
```

(1)

(2)

(3)

שאלה **4**

לא נענה

ניקוד השאלה: 10.00

שימו לב שנוספה שורת return בסוף הקוד של סעיף ב

מטריצות דלילות **(sparse)** הן מטריצות בהן רוב האיברים הם אפסים, ורק מיעוטם שונים מ-0 **(nonzeros)**.

יש מספר שיטות לאחסן מטריצות כאלה כדי לחסוך את המקום של האפסים הרבים. אחת השיטות היא להשתמש במילון בו המפתח הוא **tuple** עם הקואורדינטות של איבר **nonzero** כלשהו מהמטריצה, והערך הוא ה-**nonzero** עצמו.

להלן שתי דוגמאות עם הרישום הרגיל (רשימה של רשימות), והרישום "החוסך" באמצעות מילון:

```
>>> A = [[0,0,2,0],
...      [3,0,0,0],
...      [0,4,0,0]]

>>> A = {(0,2):2, (1,0):3, (2,1):4}

>>> B = [[5,0,6,0],
...      [0,4,0,0],
...      [0,5,0,7]]

>>> B = {(0,0):5, (0,2):6, (1,1):4, (2,1):5, (2,3):7}
```

---

5 .A נק') הפונקציה **`add_sparse_matrices(A, B, dim)`**, מקבלת שתי מטריצות דלילות **A** ו-**B** אשר מיוצגות כשני מילונים, בעלות מימדים זהים, ו-**`tuple`** בשם **dim** אשר שני איבריו הם מימדי המטריצות (מספר השורות ומספר העמודות שלהן).

הפונקציה מחזירה מטריצה חדשה שהיא תוצאת הסכום של **`A+B`**. מטריצה זו מיוצגת בפורמט של רשימה של רשימות.

דוגמא:

```
>>> dim = (3, 4)
>>> add_sparse_matrices(A, B, dim)
[[5, 0, 8, 0], [3, 4, 0, 0], [0, 9, 0, 7]]
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def add_sparse_matrices(A, B, dim):
    ans = []
    for i in range(dim[0]):
        ans.append([0] * dim[1])

    for i, j in ______(1)______:
        ______(2)______

    for k, m in ______(3)______:
        ______(4)______
    return ans
```

(1)

(2)

(3)

(4)

5) B. נק') הפונקציה **`two_diags(mat)`**, מקבלת מטריצה ריבועית **`mat`** אשר מיוצגת על ידי רשימה של רשימות (שורות). הפונקציה מחזירה **`tuple`** עם שני איברים, הראשון הוא סכום כל האברים של האלכסון הראשי (משמאל לימין) של **`mat`**, והשני סכום אברי האלכסון השני.

דוגמאות:

```
>>> A2 = [[0,0,2,0],
...       [3,0,0,0],
...       [0,4,0,0],
...       [1,0,3,0]]
>>> two_diags(A2)
(0,5)

>>> B2 = [[5,0,6,0],
...       [0,4,0,0],
...       [0,5,0,7],
...       [0,3,0,4]]
>>> two_diags(B2)
(13, 5)
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def two_diags(mat):
    if len(mat) == 1:
        return mat[0][0]

    diag1,diag2 = 0,0
    for r in ______(1)______:
        diag1 += ______(2)______
        diag2 += ______(3)______
    return diag1,diag2
```

(1)

(2)

(3)

שאלה **5**

תקין

ניקוד השאלה: 15.00

place_between

נתונה המחלקה **Lnk**:

```
class Lnk:
    empty = ()
    def __init__(self, data, next = empty):
        self.data = data
        self.next = next

    def __repr__(self):
        if self.next is Lnk.empty:
            return "Lnk({})".format(self.data)
        else:
            return "Lnk({}, {})".format(self.data, self.next)
```

כתבו פונקציה **place_between(inter, lnk_lst)** אשר מקבלת שני ארגומנטים: **inter**, שהוא מספר שלם, ורשימה מקושרת **lnk_lst**, שאבריה הם מספרים. הפונקציה מחזירה רשימה מקושרת חדשה שבה בין כל שני איברים שווים וצמודים ב-**lnk_lst**, מכניסים איבר חדש שערכו **inter**.

דוגמאות

```
>>> place_between(-7, Lnk(3, Lnk(3)))
Lnk(3, Lnk(-7, Lnk(3)))

>>> place_between(3, Lnk(3, Lnk(3)))
Lnk(3, Lnk(3, Lnk(3)))

>>> place_between(-7, Lnk(-7, Lnk(3, Lnk(3))))
Lnk(-7, Lnk(3, Lnk(-7, Lnk(3))))

>>> place_between(11, Lnk(7, Lnk(7,Lnk(6,Lnk(6,Lnk(11))))))
Lnk(7, Lnk(11, Lnk(7, Lnk(6, Lnk(11, Lnk(6, Lnk(11)))))))

>>> place_between(10, Lnk(9, Lnk(9, Lnk(9)))) # "חופפים" בין זוגות
Lnk(9, Lnk(10, Lnk(9, Lnk(10, Lnk(9)))))

>>> ling = Lnk(11, Lnk(21))  # no repetitions
>>> place_between(5, ling)
Lnk(11, Lnk(21))

>>> lynk1 = Lnk(8) # one element
>>> lynk2 = place_between(-1, lynk1)
>>> lynk1
Lnk(8)
>>> lynk1 is not lynk2
True
```

Answer:

(penalty regime: 0 %)

Reset answer

```
, data, next = empty):
data
next

):
is Lnk.empty:
nk({})".format(self.data)

nk({}, {})".format(self.data, self.next)

 ABOVE THIS LINE! #########
```

```
ter, lnk_lst):
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **6**

תקין

ניקוד השאלה: 15.00

no_twins

כתבו פונקציה **`no_twins(n, x, y)`** אשר מחזירה רשימה של כל הרשימות השונות באורך **n** המכילות את המספרים **x** ו-**y** בלבד, כך ש **y** אינו מופיע בהן פעמיים ברציפות.

דוגמאות

```
>>> sorted(no_twins(2, 5, 3))
[[3, 5], [5, 3], [5, 5]]

>>> sorted(no_twins(3, 5, 3))
[[3, 5, 3], [3, 5, 5], [5, 3, 5], [5, 5, 3], [5, 5, 5]]

>>> sorted(no_twins(4, 5, 3))[:3]
[[3, 5, 3, 5], [3, 5, 5, 3], [3, 5, 5, 5]]

>>> sorted(no_twins(4, 5, 3))[3:]
[[5, 3, 5, 3], [5, 3, 5, 5], [5, 5, 3, 5], [5, 5, 5, 3], [5, 5, 5, 5]]
```

Answer:

(penalty regime: 0 %)

Reset answer

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **7**

תקין

ניקוד השאלה: 15.00

ops_to_leaves

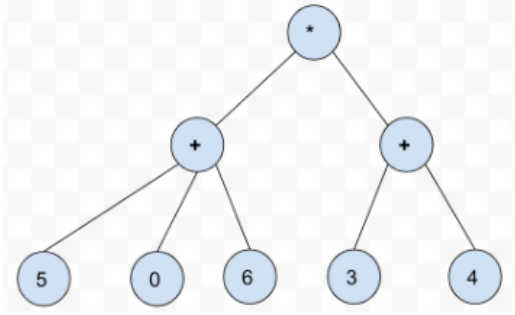
נתונה המחלקה **Tree**:

```
class Tree:
    def __init__(self, data, branches=[]):
        self.data = data
        self.branches = list(branches)

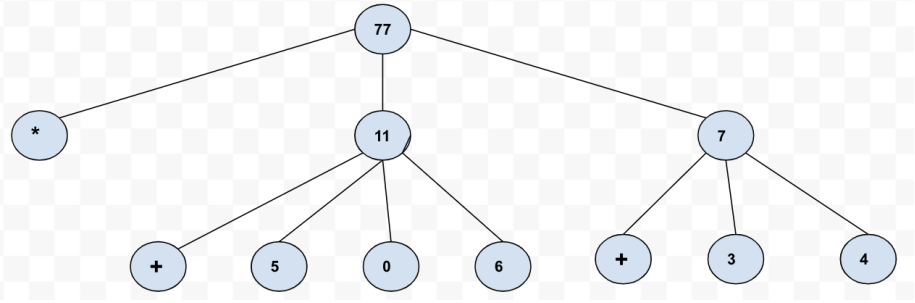
    def is_a_leaf(self):
        return not self.branches

    def __repr__(self):
        if self.branches:
            return 'Tree({0}, {1})'.format(self.data, repr(self.branches))
        else:
            return 'Tree({0})'.format(repr(self.data))
```

עץ ביטוי הוא עץ, לא בהכרח בינארי, אשר בעלים שלו נמצאים האופרנדים (הערכים של הביטוי), והאופרטורים של הביטוי נמצאים בצמתים הפנימיים שלו. דוגמא להמחשה, הביטוי **(5+0+6)*(3+4)**:



ניתן להמיר הצגה זו של עץ הביטוי, בהצגה חדשה של עץ המכיל את תוצאות חישובי הביניים של כל פעולה שבו. העץ הבא הוא המרה של העץ בדוגמא לעיל, (שימו לב לכך ש-**7,11,77** הם תוצאות של רמות נמוכות יותר):



המעבר מההצגה של עץ ביטוי רגיל להצגה החדשה, נעשה על ידי החלפת כל אופרטור בעץ הביטוי בתוצאת החישוב של בניו, והוספת אותו אופרטור כעלה שהוא הבן השמאלי ביותר (באינדקס 0) של הצומת בה תוצאת החישוב.

שימו לב שהתהליך מתבצע מתחתית העץ כלפי מעלה.

כתבו פונקציה **ops_to_leaves(iTree)** אשר מקבלת עץ ביטוי **iTree** בפורמט הרגיל, ומשנה אותו לפורמט החדש שתואר לעיל, ומחזירה את תוצאת חישוב הביטוי של -**iTree**. התוצאה תימצא בשורש העץ בפורמט החדש.

המחלקה **Tree** המצורפת לעיל מתאימה לשני הפורמטים.

דוגמאות:

```
## פורמט רגיל
>>> iTree=Tree('*',[Tree('+',[Tree(5),Tree(6)]), Tree('+',[Tree(3),Tree(4)])])
>>> print(id(iTree))
140371502523344
>>> ops_to_leaves(iTree)
77
## פורמט שני
>>> iTree
Tree(77, [Tree('*'), Tree(11, [Tree('+'), Tree(5), Tree(6)]), Tree(7, [Tree('+'), Tree(3), Tree(4)])])
>>> print(id(iTree))
140371502523344

## פורמט רגיל
>>> t1 = Tree('*', [Tree(2), Tree(3)])
>>> t2 = Tree('+',[Tree(8), Tree(7)])
>>> t3 = Tree('*',[Tree(5), Tree(7)])
>>> t4 = Tree('+',[t1, t2, t3])
>>> print(ops_to_leaves(t4))
56
## פורמט שני
>>> t4   ## פלט מחולק מסיבות טכניות
Tree(56, [Tree('+'), Tree(6, [Tree('*'), Tree(2), Tree(3)]), Tree(15, [Tree('+'),
                    Tree(8), Tree(7)]), Tree(35, [Tree('*'), Tree(5), Tree(7)])])
```

Answer:

(penalty regime: 0 %)

Reset answer

```
, data, branches=[]):
data
 = list(branches)

f):
elf.branches

(self):
hes:
ree({0}, {1})'.format(self.data, repr(self.branches))

ree({0})'.format(repr(self.data))

 ABOVE THIS LINE! #########

ee):
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **8**

תקין

ניקוד השאלה: 15.00

pardes

נתונה המחלקה **BinT**:

```
class BinT:
    empty = ()
    def __init__(self, data, left=empty, right=empty):
        self.data = data
        self.left = left
        self.right = right
```

כתבו פונקציה **pardes(lst, n=0)** אשר מקבלת רשימה: **lst** שבה מספרים וערכי **None**, ומספר שלם **n** עם ערך **0** כברירת מחדל. **n** ישמש כאינדקס ב-**lst**.

הפונקציה תחזיר עץ בינארי אשר נבנה על פי אברי הרשימה **lst** באופן הבא:

- שורש העץ נמצא ב-**lst[0]**

- הבן השמאלי של הקדקד שנמצא ב **lst[k]** נמצא ב **lst[2*k+1]**, והבן הימני שלו נמצא ב-**lst[2*k+2]**.

הערך **None** מסמן עץ או תת-עץ ריק, ואם הוא מופיע בסוף הרשימה **lst** ניתן להתעלם ממנו.

הניחו שהרשימה **lst** תקינה.

דוגמאות

```
>>> pardes([1, 2, 3],n=0)
BinT(1, BinT(2), BinT(3))

>>> pardes([5, None, 2],n=0)
BinT(5, BinT.empty, BinT(2))

>>> pardes([4, 0, None],n=0)
BinT(4, BinT(0))

>>> pardes([2, 5, 4, 6, None, 7], n=0)
BinT(2, BinT(5, BinT(6)), BinT(4, BinT(7)))
```

Answer:

(penalty regime: 0 %)

Reset answer

```
, data, left=empty, right=empty):
data
.eft
 right

 ABOVE THIS LINE! #########
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **9**

תקין

ניקוד השאלה: 15.00

how_close

נתונה המחלקה **Tree**:

```
class Tree:
    def __init__(self, data, branches=[]):
        self.data = data
        self.branches = list(branches)

    def is_a_leaf(self):
        return not self.branches
```

כתבו פונקציה **how_close(tr)** אשר מקבלת עץ אשר בצמתים שלו מספרים. הפונקציה מחזירה את ההפרש הקטן ביותר (בערך מוחלט) בין המספר שבצומת כלשהו **k** בעץ לבין סכום המספרים שנמצאים בקדקדי הילדים של **k**.

עבור עלה שערכו **t**, ההפרש יהיה: **|t-0|**.

דוגמאות:

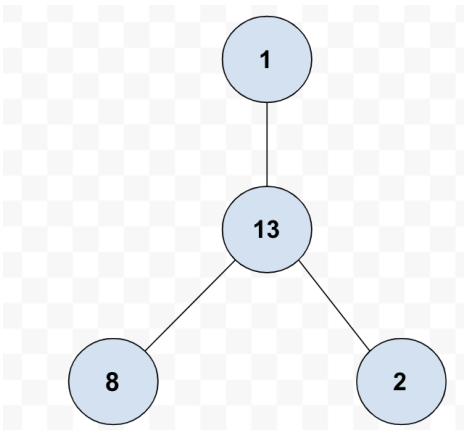
```
>>> t1 = Tree(1, [Tree(2), Tree(3)])
>>> print(how_close(t1)) # 2: |2-0| < |3-0| < |1-(2+3)|
2

>>> t2 = Tree(7, [Tree(6), Tree(5)]) # 4: |7-(6+5)| < |5-0| < |6-0|
>>> print(how_close(t2))
4

>>> t3 = Tree(13, [Tree(8), Tree(2)])
>>> print(how_close(t3)) # 2: |2-0| < |13 - (8 + 2)| < |8-0|
2

>>> t4 = Tree(1, [t3])
>>> print(how_close(t4)) # 2: |2-0| < |13-(8+2)| < |8-0| < |1-13|
2
```

תמונה להמחשה של **t4**:



Answer:

(penalty regime: 0 %)

Reset answer

```
, data, branches=[]):
data
 = list(branches)

f):
elf.branches

 ABOVE THIS LINE! #########
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **10**

תקין

ניקוד השאלה: 15.00

single_rep

נתונה המחלקה **Lnk**:

```
class Lnk:
    empty = ()
    def __init__(self, data, next = empty):
        self.data = data
        self.next = next

    def __repr__(self):
        if self.next is Lnk.empty:
            return "Lnk({})".format(self.data)
        else:
            return "Lnk({}, {})".format(self.data, self.next)
```

כתבו פונקציה **single_rep(lg)** אשר מקבלת רשימה מקושרת בה צמתים עם ערכים כלשהם. הפונקציה תשנה את הרשימה **lg**, על ידי כך שתוסיף צומת זהה אחרי כל צומת ששני שכניה (משני צדיה) שונים ממנה (ערכים שונים מהערך שלו).

הפונקציה תוסיף צומת גם אם יש רק שכן אחד, נניח בקצה הרשימה, או גם אם אין אף שכן - נניח ברשימה עם איבר אחד.

הפונקציה תחזיר את מספר הצמתים שנוספו לרשימה.

דוגמאות

```
>>> tr1 = Lnk(1)
>>> single_rep(tr1)
1
>>> tr1
Lnk(1, Lnk(1))

>>> tr2 = Lnk(3, Lnk(4, Lnk(3, Lnk(4))))
>>> single_rep(tr2)
4
>>> tr2
Lnk(3, Lnk(3, Lnk(4, Lnk(4, Lnk(3, Lnk(3, Lnk(4, Lnk(4))))))))

>>> tr3 = Lnk(3, Lnk(4, Lnk(4)))
>>> single_rep(tr3)
1
>>> tr3
Lnk(3, Lnk(3, Lnk(4, Lnk(4))))
```

Answer:

(penalty regime: 0 %)

Reset answer

```
, data, next = empty):
data
next

):
is Lnk.empty:
nk({})".format(self.data)

nk({}, {})".format(self.data, self.next)

ABOVE THIS LINE! #########
```

	Comments:
✔	Answer Saved. (No tests were run on the code at this stage. You can use the Precheck button to run your code with no penalty.) You can still edit the answer if you wish.

Passed all tests! ✔

שאלה **11**

לא נענה

ניקוד השאלה: 1.00

סמנו איזה שאלות בחרתם בחלק ב', השאלות שתסמנו הן אלו שיבדקו **בלבד**.

- שאלה 5 - place_between
- שאלה 6 - no_twins
- שאלה 7 - ops_to_leaves
- שאלה 8 - pardes
- שאלה 9 - how_close
- שאלה 10 - single_rep

שאלה **12**

לא נענה

לא ניתן ציון

תוכלו לרשום כאן הערות וטקסט חופשי במידת הצורך. **חלק זה אינו נבדק מראש** אלא רק במקרים חריגים ובמקרה של ערעורים.